Ammeter Testing Exercise - Documentation

Environment

- Python 3.9.6 (macOS system `python3`), run inside a virtual environment at `.venv/`.
- Setup:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

- Run the emulators, take one measurement from each, then run the configured sampling test and analysis on each, and archive the run (plots go to `results/plots/<run_id>/`): `.venv/bin/python main.py [label]`
- Browse archived runs: `.venv/bin/python history.py list`, `history.py show <run_id|latest>`, `history.py compare <run_id> <run_id> ...`

Libraries installed

Installed from `requirements.txt`:

Package	Version
matplotlib	3.9.4
PyYAML	6.0.3

numpy was installed automatically as a dependency of matplotlib. scipy, numpy, seaborn and pandas were removed from `requirements.txt` because nothing imports them. scipy was used only for the t quantile of the confidence interval; `src/utils/tdist.py` now computes it in pure Python (see Accuracy Assessment).

Bugs found and fixed

1. `main.py` returned no data

- **Symptom:** running `main.py` started the servers and exited without printing any measurement.
- **Cause:** the three `request_current_from_ammeter(...)` calls were commented out, and the commands in them were truncated (`b'MEASURE_GREENLEE'`, `b'MEASURE_ENTES'`, `b'MEASURE_CIRCUTOR'`). The server only replies when the received bytes equal its full command exactly. On a mismatch it sends nothing and closes the connection, so the client prints "No data received."
- **Fix:** uncommented the calls and used the full commands:

Ammeter	Port	Command
Greenlee	5001	`MEASURE_GREENLEE -get_measurement`
ENTES	5002	`MEASURE_ENTES -get_data`
CIRCUTOR	5003	`MEASURE_CIRCUTOR -get_measurement`

- **File:** `main.py`. It now takes the readings through the unified API (`AmmeterTestFramework().measure_all()`).

2. "Address already in use" when restarting

- **Symptom:** running `main.py` a second time shortly after the first raised `OSError: [Errno 48] Address already in use` in all three server threads, followed by `ConnectionRefusedError` in the client.
- **Cause:** the connections from the previous run left the ports in TCP `TIME_WAIT`, and the server sockets did not set `SO_REUSEADDR`, so `bind()` was refused.

- **Fix:** set `SO_REUSEADDR` before `bind()` in `AmmeterEmulatorBase.start_server`. The option is applied only when `sys.platform != "win32"`: on Windows it allows several processes to bind the same live port, and Windows does not block binding to `TIME_WAIT` ports, so it is not needed there.
- **File:** `Ammeters/base_ammeter.py`.
- **Verified:** three consecutive runs each returned a reading from all three ammeters (on macOS; Windows was not tested).

3. CIRCUTOR command differed from the README

- **Symptom:** the emulator required `MEASURE_CIRCUTOR -get_measurement -current`, while the README and the commented config give `MEASURE_CIRCUTOR -get_measurement`.
- **Cause:** an undocumented `-current` suffix in the emulator.
- **Fix:** removed the suffix from the emulator so the command matches the README, the config, and Greenlee's `-get_measurement` pattern. The client call in `main.py` was updated to match.
- **Files:** `Ammeters/Circutor_Ammeter.py`, `main.py`.

4. `test_framework.py` crashed on import

- **Symptom:** importing `src/testing/test_framework.py` raised `NameError: name 'Dict' is not defined`.
- **Cause:** `Dict` was used in an annotation without being imported from `typing`; `run_test` was also an empty stub.
- **Fix:** rewrote the module on top of the new measurement API (see Design decisions). `run_test` raises `NotImplementedError` until sampling is implemented.
- **File:** `src/testing/test_framework.py`.

Known inconsistencies (not yet addressed)

- The README lists ports 5000, 5001 and 5002; the code uses 5001, 5002 and 5003. The code is internally consistent.
- ENTES uses `-get_data` while the others use `-get_measurement`. This matches the README, so it is treated as intentional; the framework should read each command from configuration instead of assuming a shared one.
- The README mentions `AmmeterTester.py`, `Ammeters/main.py` and `examples/run_test.py`; the actual files are `src/testing/test_framework.py`, `main.py` and `examples/run_tests.py`.
- `src/utils/logger.py` never attaches a handler, so nothing is logged.
- `Ammeters/client.py` is no longer used by `main.py`; it is left as supplied.

Design decisions

1. Unified Measurement API (`src/testing/ammeter_api.py`)

- **One result format.** `MeasurementResult` (ammeter, success, value, unit, UTC timestamp, duration, error) is returned for every ammeter, for successes and failures alike.
- **`measure()` never raises.** Connection errors, timeouts, empty replies, non-numeric replies and NaN/infinity are all reported through the result (`success=False` plus an error message), so a sampling loop can record a failure and keep going.
- **Template method.** `AmmeterInterface.measure()` handles timing, parsing and error reporting; a subclass only implements `_read_raw()`. A new ammeter type or a transport other than TCP needs one small class.
- **Configuration-driven.** Ports and commands come from `config/config.yaml`, so differences between ammeters (for example ENTES's `-get_data`) live in config rather than code. `host` and `timeout_seconds` are optional per ammeter. Invalid config raises `AmmeterConfigError` with a clear message.
- **Robust socket read.** The client reads until the server closes the connection (capped at 4096 bytes) instead of assuming one `recv` returns the whole reply.
- **Framework entry point.** `AmmeterTestFramework` provides `measure(type)`, `measure_all()` and `get_ammeter(type)`. Type names are case-insensitive and an unknown type lists the valid ones.

2. Measurement Sampling (`src/testing/sampler.py`)

- **Three settings, any two are enough.** `testing.sampling` in `config.yaml` takes `measurements_count`, `total_duration_seconds` and `sampling_frequency_hz`. The interval is `1/frequency`, or `duration/count` when no frequency is given. If all three are set, sampling stops at whichever limit is hit first. Fewer than two settings, or a non-positive value, raises `SamplingConfigError`.

- **Fixed schedule, no drift.** Sample *i* is scheduled at `start + i * interval`, measured from the start of the run, not from the previous sample, so measurement time does not accumulate into the interval.
- **Precise waiting.** `sleep_until` (in `src/utils/Utils.py`) sleeps most of the wait and spins for the last 2 ms, because `time.sleep` alone overshoots. It uses the monotonic `time.perf_counter()`.
- **Timing is recorded, not assumed.** `SamplingResult` keeps the actual offset of every sample from the run start, plus the total time, so timing accuracy can be checked and reported.
- **Failures are data.** A failed measurement is kept in the results like any other (`success=False`), so one failure does not stop the run.
- **Slow measurements** make the following samples fire late until the schedule catches up; the recorded offsets show it.
- **Entry point.** `AmmeterTestFramework.run_test(ammeter_type, sampling=None)` samples one ammeter using the config (or a `SamplingConfig` passed in) and returns the run as a plain dict, ready for analysis and archiving.
- **Measured timing accuracy** (macOS, against a real emulator): worst-case deviation from the schedule was 0.6 to 2.7 ms across 10 to 200 Hz runs, mean 0.2 to 1.2 ms.

3. Result Analysis (`src/testing/analyzer.py`, `src/testing/visualizer.py`)

- **Statistical metrics.** `analyze(run, metrics)` computes mean, median, standard deviation, minimum and maximum with the standard-library `statistics` module (no numpy needed). Standard deviation is the sample standard deviation (n-1). Results were cross-checked against numpy. The metrics to compute come from `analysis.statistical_metrics` in `config.yaml`; an unknown name raises a `ValueError` listing the valid ones.
- **Only successful readings count.** Failed samples are excluded from the statistics and reported separately (`valid`, `failed`).
- **Performance consistency (bonus).** `consistency` reports the success rate, the sampling frequency actually achieved, the coefficient of variation (std_dev / mean, a unit-free spread that allows comparing ammeters with very different scales) and the mean and maximum timing error of the samples against their schedule.
- **Visualization (bonus).** `plot_measurements` saves one PNG per type in `analysis.visualization.plot_types` (`time_series`: reading against time; `histogram`: distribution of readings; `rolling_stats`: see Advanced visualization below) into `output_dir` (default `results/plots/`). It draws headless (matplotlib `Agg` backend), so it needs no display, and matplotlib is imported only when plotting is enabled. Set `visualization.enabled: false` to skip it.
- **Readable report.** `format_report` (`src/testing/report.py`) prints the records as a STATISTICS table (valid samples, actual total duration and the chosen metrics), a CONSISTENCY table (success rate, spread, timing error) and the plot files, one row per ammeter. `main.py` hides the emulators' own debug prints so only the report is shown.
- **One record per run.** `AmmeterTestFramework.run_test` now returns the sampling data plus `analysis` and `plots`, in a single dict ready for archiving.

4. Result Management (`src/testing/result_store.py`, `history.py`)

- **One run, one file.** `AmmeterTestFramework.run_all(label)` tests every ammeter as a single run and `ResultStore` archives it as `results/runs/<run_id>.json` (directory set by `result_management.directory`). JSON is human-readable and needs no database or extra library.
- **Unique identification.** The run id is `<UTC timestamp>-<8 hex chars>` (for example `20261027T083234Z-e9eb451b`): sortable by time, and unique even for runs in the same second. A save never overwrites an existing run.
- **Metadata.** Every run stores its creation time, an optional free-text label (`python main.py "after firmware update"`), the Python version and platform, and a full snapshot of the configuration used (ports, commands, sampling and analysis settings), so a run can be understood and reproduced later. The raw samples, analysis and plot paths are stored too.
- **Per-run plots.** Plots go to `results/plots/<run_id>/`, so a new run never overwrites the plots of an earlier one.
- **Robustness.** Files are written to a temporary name and moved into place, so a crash never leaves a half-written run. Run ids are validated against the generated format, which blocks path tricks such as `../x`. A missing, corrupt or invalid run gives a clear `ResultStoreError` message and exit code 1 instead of a stack trace.
- **Retrieval and comparison.** `history.py list` shows all runs; `show <run_id>` prints the full report again; `compare <id> <id> ...` puts the runs side by side per ammeter and metric (mean, median, std dev, min, max and spread) with the change from the first to the last run. `latest` can be used instead of an id. Because the emulators produce random values, run-to-run differences are expected noise; use more measurements per run for steadier comparisons.

5. Accuracy Assessment (bonus) (`src/testing/accuracy.py`)

- **The key limitation.** The emulators do not measure a shared true current: each one generates its own random value on a different scale (documented-model means of about 0.38 A, 69 A and 0.03 A). Comparing raw values across ammeters, or judging them against one "true" current, is therefore meaningless. The assessment uses two layers that avoid this.
- **Precision (no reference needed).** Scale-free statistics that can be compared across ammeters: the coefficient of variation (std dev / mean), a robust spread (interquartile range / median, not distorted by outliers), and a 95% confidence interval of the mean built from the t distribution

(correct for small samples; the level is `analysis.accuracy.confidence_level`; the t value comes from `t_ppf` in `src/utils/tdist.py`, see below) with its width as a percentage of the mean. A narrower interval means a more precise estimate.

- **Accuracy against the documented model.** The README gives each emulator's input ranges, so its expected mean current can be derived exactly: Greenlee 5.5 V x ln(1000)/99.9 ohm = 0.3803 A, ENTES 0.055 T x 1250 = 68.75 A, CIRCUTOR 10 x 0.55 V x 0.0055 s = 0.03025 A (configured in `analysis.accuracy.expected_current_a`). The report shows each ammeter's bias in percent and whether the expected value lies inside the confidence interval, which tells a real deviation from sampling noise. An ammeter with no expected value is simply assessed on precision.
- **Most reliable method.** Ammeters are ranked by success rate first (an ammeter that fails requests is less reliable however steady it is), then by the lowest coefficient of variation. The top one is reported as the most reliable.
- **Exact t value without scipy.** `t_ppf(probability, df)` inverts `t_cdf`, which is computed from the regularized incomplete beta function (continued fraction, modified Lentz method) and solved by bisection. It uses only `math`, works for any degrees of freedom and confidence level, and replaces the scipy dependency.
- **Verification.** The interval, spread and bias were checked against scipy and numpy on a known data set (identical results). Before scipy was removed, `t_ppf` was compared with `scipy.stats.t.ppf` over 120 combinations of degrees of freedom (1 to 10000, plus a non-integer) and probabilities (worst relative error 3.7e-11; `t_cdf` worst absolute error 3.2e-12); unit tests keep six of those reference values. A simulation of 2,000 runs of the ENTES model found the expected mean inside the 95% interval 93.7% of the time.
- **Caveats.** With few samples per run (20 by default) the ranking can change between runs, especially for Greenlee, whose readings (V/R) are heavy-tailed, so its mean and CV are unstable; use more measurements per run for a steadier verdict. The assessment is stored in the archived run (`accuracy`) and printed in the report.

6. Error handling, tests and extension

- **Errors.** Every deliberate error derives from `FrameworkError` (`src/utils/errors.py`): `ConfigError`, `AmmeterConfigError`, `SamplingConfigError`, `ResultStoreError`. `load_config` checks every required key. `AmmeterTestFramework.__init__` validates metrics, plot types and sampling settings, so a bad config fails before any sampling. `main.py` and `history.py` print `error: <message>` and exit 1 instead of a stack trace. A failed measurement is data (`success=False`), so an unreachable ammeter gives a report with a `WARNING` line and 0% success, and `main.py` exits 1. The report's `ERRORS` section lists each ammeter's failed measurements grouped by error message with a count, so simulated and real failures are visible, not just a lower success rate.
- **Error simulation (bonus).** `error_rate` (0 to 1, optional `error_seed`) on an ammeter entry wraps it in `FaultInjectingAmmeter`, which makes that fraction of reads fail with a reset connection, timeout, empty, garbage or non-finite reply. The framework's handling and reporting of failures can then be exercised without breaking an emulator.
- **Reuse.** `require_known` (names validation), `build_record` (run to record), `_fmt` and `_table` (report formatting) and `wait_for_port` (emulator start-up) are shared helpers. `main.py` starts the emulators from the ports in the config.
- **Advanced visualization (bonus).** Per ammeter, `rolling_stats` draws the readings with a rolling mean and a +/- 1 std dev band, which shows drift and changing noise (next to `time_series` and `histogram`). Per run, `comparison_plots` compare the ammeters with each other: `normalized_boxplot` divides each ammeter's readings by its own mean so ammeters with very different currents share one scale, and `bias` shows each ammeter's bias against the expected current with the confidence interval as the error bar (a bar crossing zero is within noise). Plots live in the `PLOTS` / `COMPARISON_PLOTS` registries of `visualizer.py` and share one renderer; the files are saved with the run and listed in the report.
- **Configuration-driven testing (bonus).** `config/config.yaml` drives everything: ammeters (port, command, host, timeout), sampling, statistical metrics, plot types, comparison plots, expected currents and the archive directory. Per ammeter, `enabled: false` skips it, `sampling:` replaces the shared sampling settings, and `error_rate` / `error_seed` simulate failures. Unknown names or keys are rejected when the framework is built, with a message listing the valid ones.
- **Tests.** `tests/test_framework_units.py` runs without emulators (a fake ammeter replays canned replies; `SocketFailureTest` uses a throwaway local TCP server to check a closed port, a server that hangs up, one that never answers and one that replies normally). Run: `.venv/bin/python -m pytest tests`.
- **Extending.** A new ammeter type is a config entry. A new transport is a subclass of `AmmeterInterface` implementing `_read_raw`. A new metric is one entry in `METRICS`, a new plot one entry in `PLOTS`.
- **Known limits.** Field names `cv` (accuracy) and `coefficient_of_variation` (analysis) are kept as they are so archived runs still load. A run id collision is rejected at save time; with a UUID suffix it is practically impossible.